8INF878 – Intelligence artificielle agentique
Travail No. 1
AUSSI V2 – Résolution de problème et généralisation

Mise en situation :
Vous devez développer une nouvelle version de AUSSI — Agent Universel de Support Super Intelligent. Jusqu'à maintenant, AUSSI a appris à résoudre des problèmes simples. Votre objectif est maintenant de concevoir son moteur générique de résolution de problèmes. AUSSI devra être capable de résoudre des problèmes très différents sans modifier ses algorithmes d'exploration. Seule la formulation du problème et, lorsque nécessaire, son heuristique pourront changer. L'objectif principal du travail n'est donc pas de programmer quatre solutions indépendantes, mais de construire un agent généraliste de résolution de problèmes.

Les problèmes qu'AUSSI est capable de résoudre :
1. Labyrinthes. AUSSI doit trouver un chemin entre une position initiale S et une destination G dans un labyrinthe comportant des obstacles. Les labyrinthes peuvent être simple, moyen, complexe. Ils peuvent avoir un coût uniforme, mais certains obstacles peuvent changer la donne. Vous devez gérer des types de labyrinthes différents.

2. Navigation. On donne à AUSSI un graphe représentant des villes et des routes pondérées. Chaque ville possède des coordonnées géographiques simplifiées. AUSSI doit déterminer un itinéraire entre deux villes. C'est ton « mini Google Maps ». L'heuristique peut naturellement être la distance à vol d'oiseau, mais devrait pouvoir être modifiée sans causer de problèmes majeurs (en respectant admissibilité et consistance).

3. N-Puzzle. AUSSI peut s'attaquer au N-Puzzle. Prévoyez le cas où la résolution serait trop longue voir risquerait de dépasser les capacités en mémoire d'AUSSI. Peut-être que le robot se met lui-même en arrêt de travail?

4. Équations / jeu arithmétique. AUSSI devrait pouvoir résoudre des équations simples et nous donner la trace de son raisonnement. Par exemple, à partir des chiffre 2,3,5,7 et des opérations $+ - \times \div$, construire l'expression dont le résultat vaut 24. Attention! Certains types de problèmes n'ont pas de solution…

Contraintes à respecter :
1. Implémentation minimale de BFS, UCS, Greedy Search et A*. Gardez en tête qu'en principe je devrais pouvoir ajouter une stratégie en changeant très peu le code.
2. Possibilité de changer entre Tree-Search et Graph-Search sans réimplémenter les algorithmes → SEARCH(problem, strategy, mode)
3. Avoir une interface qui permet à l'utilisateur de choisir le problème, la stratégie et le mode. Pensez-y-en terme évolutif… éventuellement un LLM remplacera votre interface…
4. AUSSI doit minimalement dire si une solution a été trouvée et la présenter, donner le coût et la profondeur ainsi que quelques statistiques (noeuds générés, noeuds expansés, taille max OPEN, temps : 0.083 s, etc.)
5. Vous devez produire une petite expérience comparative des algorithmes sur vos problèmes.

L'utilisation d'outils d'IA générative est autorisée. Vous êtes responsables de l'intégralité du code remis, peu importe son origine. Lors de l'évaluation, chaque membre de l'équipe pourra

être interrogé individuellement sur n'importe quelle partie du travail. Une fonctionnalité que l'équipe est incapable d'expliquer pourra être considérée comme **non maîtrisée**, même si elle fonctionne correctement.

Vous serez évalués sur les trois aspects suivants :

1. La qualité de votre travail (fonctionnalité, complétude, généralisation, qualité du code)
2. Votre compréhension à l'entrevue
3. La qualité du rapport (capacité de synthèse, qualité de la réflexion et des analyses)

Date limite de remise : **9 octobre 2026 à 23h59**
Travail en équipe de 3 à 4.

Bon travail !